

Drone Project

Group: 19

Name	Student Number
Borislav Semerdzhiev	5578043
Martijn Leeuwenburg	5819148
Alireza Saghaeian	6450180
Daniel Hernández García	6625665

April 12, 2026

Abstract

This project implements an embedded Rust control system for a quadcopter drone. Our approach is based on a finite-state machine with safe, calibration, manual, full-control, yaw-control, raw-control, height-control and panic modes to operate the drone. The drone receives commands from a PC via UART through an HDLC-based communication protocol. However, we also achieved wireless communication via Bluetooth Low Energy. The PC sends to the drone requests when changing to a different mode in the FSM, as well as the different inputs from the keyboard and joystick. On the drone side, it sends telemetry data such as the current mode, battery, motor values, sensor readings, and the code execution time. The resulting implementation provides a modular approach for manual flight and stable drone behavior. Overall, the project demonstrates how Rust can be used to build a structured and memory-safe drone control architecture on bare-metal embedded hardware.

1 Introduction

1.1 Background

Embedded systems play a central role in modern technology by combining software with programmable hardware to perform dedicated, real-time tasks. In many practical applications, microcontrollers and other programmable components are preferred over custom hardware because they offer greater flexibility, faster development, and lower cost. As a result, embedded systems design often focuses not only on hardware integration, but also on reliable embedded software that can interact with sensors, actuators, and communication interfaces under strict timing and safety constraints. In this course, the focus is on programmable embedded hardware and the software needed to control them. A quad-rotor drone is a suitable case study because it combines sensing, actuation, communication, and control in one system.

1.2 Problem Statement

The aim of this project was to implement embedded software for the TU Delft quadrupel drone. Since a drone is naturally unstable, it requires continuous corrections and control to achieve stability during flight. This means that sensor data must be processed quickly and translated

into motor commands while also making corrections to counteract external noise that would destabilize the drone.

The problem statement of this project is therefore: How can a safe and modular embedded software system be designed to control and stabilize a quad-rotor drone? To address this, the project implements mode-based control, sensor calibration, communication over UART (and Bluetooth), and motor control based on feedback. Specifically, the system must achieve both the safety and functional requirements. Namely:

- **Safety Requirements:** Implement a safe state, emergency stop at any time, battery check, robustness against noise, reliable communication, handling sudden disconnection.
- **Functional Requirements:** Implement Safe Mode, Panic Mode, Manual Mode, Calibration Mode, Yaw-controlled Mode, Full-control Mode, Raw Sensors Full-control Mode, Height-control Mode and Wireless Mode (optional but implemented regardless).

2 Architecture

2.1 System Overview

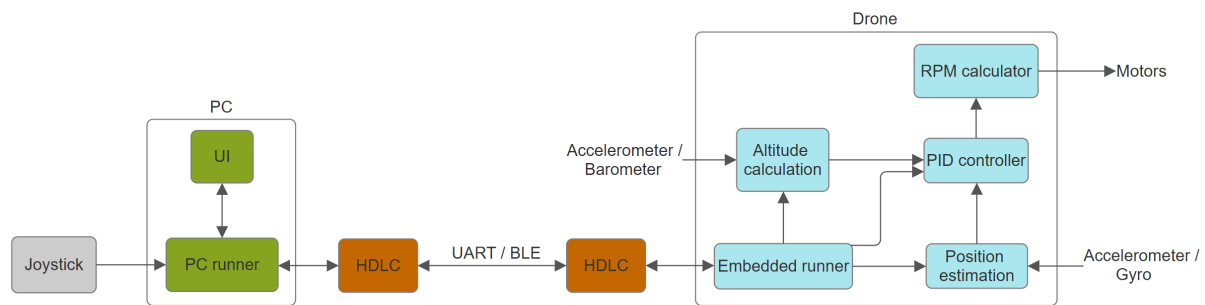


Figure 1: Functional block diagram of the overall system architecture.

The system is composed of the following major components:

1. **PID Controller** — Calculates the correction that needs to be fed to the RPM calculator.
2. **RPM Calculator** — Using the values provided by the PID controller, calculate the PWM signals that will actuate the motors.
3. **Position Estimation** — Performs a Kalman Filter (Simplified) on the accelerometer and gyro raw readings, in order to estimate the drone position.
4. **Altitude Calculation** — Uses a Kalman filter on the barometer and the accelerometer to estimate the current altitude of the drone.
5. **HDLC Transceiver** — Library used by both the drone and pc side for sending variable sized messages.
6. **Drone->PC Communication** — Establishes communication in the direction Drone to PC.
7. **PC->Drone Communication** — Establishes communication in the direction PC to Drone.
8. **Joystick and keyboard reader** — Reads joystick and keyboard.
9. **PC<->UI Communication** — Connection between the PC and the UI to display data.

10. **UI Display** — UI that displays drone telemetry and enables dynamic changes to the PID values.
11. **Bluetooth Low Energy** — Enables Bluetooth Low Energy (BLE) and receives/sends packets.

An overview of the overall system can be found in Figure 1.

2.2 Component Descriptions

2.2.1 PID Controller

The responsibility of the PID controller is to compare the **current** position of the drone with the **target** position, provided by the joystick, and actuate the motor in a specific way as to correct any differences.

The PID controller internally stores the previous error and an integration build up, that is used when adding the I part. Every iteration of the main loop, the drone samples its current position (either from DMP or through raw readings) and calculates the error by subtracting the joystick input from its position.

The PID uses three different parameters for each control axis plus height, that correspond to corrections from the present, the past, and the future. In order:

- P(present) - Multiplies the difference between the joystick input and the sensor sample.
- I(past) - Multiplies the integration build up of previous errors.
- D(future) - Multiplies the difference between the current error and the previous error.

These 3 terms are at the end added together, and the correction is forwarded to the RPM calculator.

During the design stage, the team considered using cascaded PID controller, but this idea was rejected due to the tuning complexity that comes with it, and the little gain that it would bring, as the drone is not going to be flown outside.

2.2.2 RPM Calculator

The objective of the RPM calculator is to process the control commands (roll, pitch and yaw) and use them to get the appropriate RPMs (Revolutions Per Minute) for each of the four motors from the drone. To achieve this we need to use the following quadrotor mixing equations:

$$Z = -b(\Omega_1^2 + \Omega_2^2 + \Omega_3^2 + \Omega_4^2), \quad L = b(\Omega_4^2 - \Omega_2^2), \quad (1)$$

$$M = b(\Omega_1^2 - \Omega_3^2), \quad N = d(\Omega_2^2 + \Omega_4^2 - \Omega_1^2 - \Omega_3^2) \quad (2)$$

In these formulas " Z, L, M, N " refer to the desired lift, roll, pitch and yaw. The b and d are the thrust and drag coefficients, while the Ω refer to the RPM of each corresponding motor (1-4). Once the corrected control signals have been calculated by the PID controller, these values are transformed into Fixed-point numbers since we need to use them for the calculation of the above mentioned equations as the drone lacks a floating point unit.

In order to solve for each Ω^2 we used Gaussian elimination:

$$\Omega_1^2 = -Z + 2M - N, \quad \Omega_2^2 = -Z - 2L + N, \quad (3)$$

$$\Omega_3^2 = -Z - 2M - N, \quad \Omega_4^2 = -Z + 2L + N \quad (4)$$

After substituting the values in the equation above we compute the squared RPMs of each motor and subsequently apply the square root to find the final RPMs. Finally the RPM calculator transforms the RPMs into PWM (Pulse Width Modulation) values to feed the motors directly.

2.2.3 Position Estimation

Although the DMP sensor provided the drone's position in the form of quaternions, this method was a major bottleneck in the drone's operation. An alternative method is to read the raw gyro and accelerometer data and calculate the drone's position. However, the raw sensors are susceptible to noise and drift. In order to fix each issue, a combination of a first-degree Butterworth filter and a simplified Kalman filter was used.

The Butterworth filter was used to filter the initial noise, and afterwards, the Kalman filter predicts the roll and pitch using the gyro's current reading, the last recorded position, and the bias. Then, it uses the accelerometer's current reading to correct the gyro's estimation and update the system's bias for further reading.

This system was used for estimating roll and pitch, while the yaw just used the filtered gyro's Z value to update the drone's position.

2.2.4 Altitude Calculation

Altitude estimation of the drone is needed in order to make height control work. Two available sensors can be used - the barometer and the accelerometer. The barometer provides a stable over time, but noisy readings, and the accelerometer provides accurate readings, that however drift in time as the sensor reading is integrated to get meters from ms^{-2} .

Kalman filter combines the readings of the two sensor to produce a stable output.

The internal structure of the class is the following:

```
current_state : Matrix2x1,   observation_matrix : Matrix1x2,   uncertainty_matrix :  
Matrix2x2,   barometer_variance : scalar,   accelerometer_variance : scalar,  
last_reading_accel : Instant,   last_reading_baro : Instant
```

The algorithm implemented for the Kalman filter is the standard one, in which multiple predictions are done and the uncertainty grows with each next one, and when a correction is performed, Kalman gain is calculated and used for the update of the uncertainty matrix and the newest reading.

The nalgebra crate was used to facilitate the implementation of the matrix multiplication, however, due to incompatibility of nalgebra and fixed, some matrix multiplications had to be done by hand - in other words, written by hand first and translated to code.

Furthermore, due to numerical error issues after transitioning to fixed point, Joseph's form was used for the update equation. It is numerically more stable, but requires additional computation time. This is a trade-off that was worth making.

2.2.5 HDLC Transceiver

The HDLC library is used for serializing/deserializing variable sized messages that are sent between the drone and the PC. It allows for quick error recovery after a byte is corrupted/dropped by performing CRC computation and frame boundary detection. Each message contains boundary frames at the beginning and at the end, and whenever a byte in the payload has the same value as the frame byte, an escape byte is prepended before it and the corresponding byte is XOR-ed with a previously configured byte. This is done to ensure that whenever a frame boundary byte is received, it should mark either the beginning or the end of a message. Additionally, any byte

that has the value as an escape byte in the payload of the message is also escaped. At the end of the message right before the end frame byte, a 32 bit CRC is added.

Internally, the library consists of three ring-buffer arrays: one in which the newly received bytes are inserted, and the two others are used as helper arrays when deserializing a message.

An important design decision is the size of the main buffer. The chosen values are 2^{15} on the PC side and 2^8 on the drone side. Having a small buffer carries the risk of dropping messages, as the policy used in the library is to overwrite bytes if the ring-buffer is full, but having a large buffer is too expensive for the limited amount of RAM the drone has. Therefore, 2^8 bytes is precisely the sweet middle on the drone side.

2.2.6 Drone->PC Communication

The drone to PC communication was used mainly for periodic telemetry data and drone status updates. Depending on the communication channel, the drone periodically sends its battery level, current mode, sensor data, position, and motor speed to the PC. This data is then used in the GUI to display the drone's information and provides a good method for the PC to detect drone disconnection.

The challenge was to figure out a method to send the telemetry data without any buffer overflow happening, which was fixed by dividing the telemetry data into smaller chunks and sending each chunk before generating the next one.

2.2.7 PC->Drone Communication

The PC to drone communication was used for sending the control data extracted from the joystick and keyboard to the drone. These controls included the new position, mode change request, and the PID controller's new parameters. The packets were encapsulated in the `DeviceCommand` enum. The state change request first checks if the transition is possible or not, and then sends the packet to the drone.

The drone uses the frequency of `ManualInput` received to ensure that the connection between the drone and the PC is established, and it would switch to panic mode if it hasn't received any new packets after a while.

2.2.8 Joystick and keyboard reader

The joystick and keyboard reader are responsible for the human input of the controller. Both input sources are read in a non-blocking manner, so the program can continuously monitor commands without stopping other communication or UI tasks. The joystick provides the main flight actions for lift, roll, pitch and yaw, while the keyboard is used for trim adjustments, mode changes, and emergency actions such as triggering panic mode. The values from both peripherals are combined into a single control command and transmitted to the drone at a fixed interval of 100ms.

2.2.9 PC<->UI Communication

For this project we used a python graphical user interface to support control of the drone and display the information received from it. To achieve this, the runner code connects to the GUI through a Unix domain socket. On the PC side, incoming telemetry from the drone and outgoing user input are serialized into JSON and forwarded through the socket. On the UI side, a dedicated reader thread continuously receives this JSON data, parses it and stores the results for visualization. Communication also works in the opposite direction, with the UI periodically sending updated PID trim values back to the runner code and forwards them to the drone.

2.2.10 UI Display

The UI display was implemented through python libraries like Dear PyGui for the main dashboard and PyQtGraph for drone 3D visualization. The interface displays general information such as the current drone FSM mode, communication packet size, communication mode, packet delay, joystick position and PID trim values. Our UI also consists of other windows that display drone live and logged feed as well as message logs between the drone and PC. As for live feed, graphs are provided to show the change in time for variables like pressure, roll, pitch and yaw. It is worth mentioning that the separate displayed readings are for both when using the DMP and when using our filtering.

In addition to the dashboard, the UI includes a 3D drone visualization. This visualization is updated continuously using the received roll, pitch, and yaw information which shows the orientation of the drone in a more intuitive graphical form. The motor values are visualized as well, helping the user relate attitude changes and actuator outputs. Together, the dashboard and 3D visualization provide both detailed numerical and visual insights of the drone's behaviour.

2.2.11 Bluetooth Low Energy

The nRF51822 board supports a proprietary protocol called SoftDevice110 for BLE communication, when SoftDevice is flashed onto the board's memory. However, there is no Rust crate that provides an API access out of the box to this protocol for this version (newer versions like the nRF52 are supported). To fight this limitation, Nordic's SDK written in C is directly compiled and added as a static binary to the compiled Rust code. Using the SDK, three additional functions are declared and exported to Rust for use - one for setting up BLE, one for sending a packet over Bluetooth, and one that gets called after a packet has been received.

This set-up enables for seamless integration with BLE, but there are some caveats. Changes to how the Peripherals are instantiated in the Rust bare-metal code had to be done, specifically changing the priority of certain interrupts (since SoftDevice requires that there are no interrupts of priority 0, 1), and the lower memory banks had to be turned on, as per BLE requirements.

The process of trial and error was quite extensive, due to the incompatibility of bare-metal C and Rust code, and the fact that both sides wanted to be the one to perform the set-up of the system registers. The result though, low latency ($\sim 70ms$ for acquiring the channel) wireless communication over BLE, that requires just a single board and minimal changes to the code on Rust's side.

3 Implementation

3.1 Technology Stack

Technology	Purpose
Rust	Used for implementing the bare-metal code for the drone and additionally the code for the PC runner.
Python	Used for implementing the UI. More specifically, the Python dependencies that are used are DearPyGUI, PyOpenGL and PyQt6.
C	Used for setting up, receiving, and transmitting BLE packets. Rust does not have a crate supporting SoftDevice110, so C had to be used along with the nRF5 SDK.
nRF5 SDK	Software Development Kit (SDK) provided by Nordic. Used for implementing wireless communication.
SoftDevice110	BLE proprietary protocol that can be flashed onto the board memory.

3.2 Individual Contributions

This section of the report describes the individual contribution of each team member for the project.

3.2.1 Borislav

Throughout the project, I worked on multiple parts of it. At the beginning, I had to make the communication between the PC and the drone work, and make sure both sides can recover from a dropped/corrupted byte. Later on, I moved on to designing the function that accepts the rotation rates and calculates the motor PWM signals to output.

After that, I re-implemented the PID controller, extending it beyond only using a P value to also include both I and D factors, and refactored its associated states, making them have a modular robust design that reduces code duplication.

Afterwards, I worked on the Kalman Filter for the altitude estimation, and to verify that all these modes are producing correct output, I added tabs in the UI plus 3D visualizations, and code for sending telemetry for them to the PC. I ended up implementing a large portion of the UI.

After the altitude estimation was done, I implemented the height control mode. My contribution also includes replacing all the floating point calculations in our code base with fixed-point representations, and verifying that the precision loss is acceptable. This helped me increase the base speed of our main loop from $10ms = 100Hz$ to $3ms = 333Hz$.

Furthermore, on the runner side, I replaced the single super-loop we initially had with multi-threaded approach, ensuring no deadlocks or data races can occur, by utilizing producer/consumer design patterns.

Finally, I took on the challenge of implementing wireless mode over Bluetooth, motivated by the fact that no previous team has made it work with just one board, and after long hours of implementing/reading online/debugging, made the system work correctly.

3.2.2 Alireza

At the start of the project, I contributed to adding a CRC check to the HDLC transceiver and worked on designing the packets used for communication. Then I moved to designing the structure of the drone's FSM using dynamic dispatch, which reshaped the main control loop of the drone. Afterwards, I worked on the safety checks on the drone and the runner.

Furthermore, I started working on the telemetry data and storing it on the flash memory. While this idea was refactored into periodically sending chunks of the data, the original idea is reused in the wireless mode.

After refactoring the code to use dynamic dispatch for the sensor readings, I worked on refactoring the existing Butterworth filter to use the new method and implemented the simplified Kalman filter for the drone as well. Since I already worked on raw mode, I started working on the conversion to fixed point, and implemented and tested different approximate functions such as atan and square root based on the Newton-Raphson method. During the final refactor, the team decided on using the existing Cordic library, but kept the approximate square root function.

Finally, I spent most of the lab time testing and tuning the PID controller for all modes.

3.2.3 Martijn

At the start of the project, I worked on connecting the flight control stick to the PC and additionally provided all the necessary command inputs to be used in the initial development of the manual mode. After that I worked on implementing safety features and the yaw control to be used in the PID controller.

Later, I worked on the PC to UI communication and testing live values sent from the drone on the UI, by virtue of implementing the necessary information parsing.

Finally, I worked on an initial version of wireless mode, to be used with the radio communication of 2 boards. Because of resource constraints (a maximum of 2 available boards, with one always in use to test other important features), the decision was ultimately made to switch the working of wireless mode to use a single board over Bluetooth.

3.2.4 Daniel

At the start of the project, I was initially tasked to work on Calibration mode. In the algorithm, the raw accelerometer and gyro values are read and continuously accumulated as a sum during each iteration of the control loop. After 5 seconds of the drone being in calibration mode and accumulating values, it stops reading values and then calculates a final average by dividing both sums by the number of samples taken, which is the defined as the calibration offset. This offset is crucial since it's used in the rest of modes to determine a reference point of the initial drone sensor values. Additionally, I also implemented unit tests to ensure that the calibration offset is calculated and removed accordingly.

After this I was assigned to work on pitch and roll control for Fullcontrol mode. In the control loop from this mode the DMP sensor readings are transformed from quaternion into fixed point and later used to calculate the corrected control signals for the P controller. I also worked on a function that computed the squared RPMs shown in the quadrotor mixing equations, although it is worth mentioning that this calculation was further fixed by my teammates later on. Aside from this, I coded unit tests that checked the correct calculation of the pitch and roll in the P controller as well as the computation of the squared RPMs based from the corrected pitch and roll control signals.

Finally, I helped working on the GUI and achieved an interface that displayed the current drone FSM state, battery level, P-values, accelerometer and gyro raw sensor values, the joystick inputs and the motor outputs. Moreover, I added graphs showcasing how the drone's roll, pitch and yaw changes over time, which worked very smoothly. For debugging a message log was also added in the interface which showed all the messages between the runner and the drone. This python-based GUI proved to be really helpful when I was debugging time communication issues that prevented drone state changes. The GUI was later improved and expanded by my team.

4 Experimental Results

4.1 Demonstrated Capabilities

The following capabilities were verified in the final demonstration:

1. **Safety Features:** All of the safety features worked as expected. Disconnecting the joystick, unplugging the board from the PC, unplugging the battery, pressing the ESC/space buttons, all put the drone in Panic Mode. Furthermore, transitioning into any modes whilst the joystick/keyboard were not zeroed, and the throttle was also not zeroed, was shown to be forbidden.

2. **Calibration:** The drone correctly gathers sensor data for 5 seconds, and after the timer expires, zeroes its pitch/roll/yaw with the average of the collected data.
3. **Manual Mode:** All of the joystick inputs were shown to be mapped to correct motor outputs, and the threshold required for the lift was verified to be accurate. Direction for all the commands was shown to be accurate.
4. **Yaw/Full Control:** The drone was shown to correct its position when disturbances to its position were applied, and no oscillations were observed. Joystick inputs corresponded to correct motor outputs.
5. **Raw Control:** Same as above, no oscillations were observed and perfect response to position changes was shown.
6. **Height Control:** Height control was tested, and motors' RPM correctly increased/decreased as the offset from the starting position changed. Positive remarks for the implementation of our height control were made, stating it works really well.
7. **Wireless Mode:** All of the modes above were re-tested with wireless mode turned on, and all of them were shown to work correctly. Minimum input latency was introduced, but no remarks from the demonstrator were made about it.

4.2 Quantitative Results

The reported time values are calculated as an average over time. The main loop time is directly reported inside our UI, under the general info tab.

Table 1: Summary of experimental results.

Metric	Target	Achieved
Main Loop Speed	200Hz	333.3Hz
Position Calculation Speed	<2ms	~ 1.8ms
RPM Calculation Speed	<2ms	~ 0.7ms
PID Correction Speed	<2ms	~ 0.5ms
Rust Code Size	< 256KB	87.520KB

4.3 Known Limitations

- The PID values can only be dynamically changed from the UI when the drone is in UART mode (BLE has packet limit of 20 bytes that directly causes this limitation). A workaround would have been to send the values either only when they are changed or less often, but the decision we made was to only allow changes when in UART mode.
- Bluetooth communication introduces a bit of latency, since gaining access to the channel requires channel contention, which can take up to 70ms. A solution for this problem would be to send larger messages that contain more joystick samples in one packet. This would increase the throughput and hide the latency.

5 Conclusion

5.1 Design Evaluation

The final result is a modular code base, both on the drone's as well as on the runner's side, that supports extension in new modes and new communication protocols. One of the better

decisions we did was breaking up the telemetry data into multiple messages, as the bottleneck of the system turned out to be not the communication latency, but the memory on the drone's side.

The PID controller performs well in the indoor environment in which it was tested and designed for, but if the drone is to be flown outside, a cascaded PID controller would be preferred, since it is known to have a much better response in harsh wind conditions, though, this controller would introduce more difficulties in the tuning process.

5.2 Team Results

All objectives stated in Section 1.2 were met in the project. This includes the optional wireless mode, which although it still has a few limitations, performs the desired functionality.

5.3 Individual Reflections

Borislav The biggest win I had was making wireless communication over Bluetooth work, as it required a lot of research, trial and error and SWD debugging. The approach was certainly not a mainstream one, therefore, success was not certain. If I had to do anything differently, it would be defining the architecture of the system on the PC side earlier on. This would have saved me a lot of time spent on refactoring the code and splitting it into separate threads.

Martijn It was interesting to apply some knowledge I already gained from my bachelor (electrical engineering) to an embedded device, particularly in control theory, since it has the most visible effect on the physical device. I was also curious about several components handled by other team members and they implemented them in such a way that following their progress helped me understand the system more holistically. Overall, I am satisfied with the experience I gained from the project as a whole.

Alireza The most interesting part of this project was the hands-on experience I gained, which resonated well with my background in electrical engineering and gave me a glimpse into my future as an embedded systems engineer. If I were to start this project from scratch again, I would spend more time designing the system on paper and try to divide tasks better and convince the team to work on tighter deadlines.

Daniel The most useful and entertaining aspect of the project for me was the implementation of the Rust application on the embedded hardware and seeing it affect the drone's behavior appropriately. I have a Computer Science background so this is the first time I have had such an in-depth project where I had to directly develop the embedded software of a device, in this case a UAV which I find fascinating. The learning curve of this project was quite steep for me so if I could have studied further quadrotor theory (and not just from the slides), it would have helped me understand much better certain aspects of the drone's attitude at an earlier stage of the project. Overall, I found this project very formative as it reinforces my Rust skills in a more practical environment where I can directly see its effect on hardware.

5.4 Learning Experience

The project provided the team with a wide range of opportunities to learn how to integrate skills from different backgrounds into an embedded device. The team gained experience in basic control theory and signal processing, and expanded their knowledge on embedded programming. At the start of the project, unexpected delays raised concerns about meeting the required deadlines. However, the team was able to overcome these issues and deliver a fully realized project by the final deadline, improving their collaborative and project management skills in the process.